

Оглавление

Введение	2
Анализ предметной области. Декомпозиция задач	3
Оценка объема работ по задачам	9
Планирование разработки.....	11
Проектирование в конструировании.....	13
Use Case диаграмма.....	13
ER диаграмма	13
Диаграмма последовательностей	16
Обоснование выбора процедур процесса конструирования	18
Язык и платформа разработки	18
Инструменты	18
Технологии	18
Техника кодирования	19
Прототип приложения	20
Итоги разработки	24
Тестирование в конструировании	27
Отчет по тестированию.....	39
Приложение 1	43
Листинг основного проекта	43
Листинг модульных тестов.....	Ошибка! Закладка не определена.

Введение

Темой курсовой работы является разработка приложения для составления расписания студентов.

Целями курсового проекта являются: улучшение навыков проектирования и реализации серверных приложений, развитие компетенций решения задач в веб разработке, улучшение навыков разработки и тестирования программного обеспечения (ПО), автоматизация процесса составления расписаний.

Задачи: декомпозиция и оценка объема работ, планирование и конструирование системы, а также её последующая разработка и тестирование.

Объектом исследования является процесс конструирования ПО, а предметами исследования – планирование и разработка курсового проекта для автоматизации процесса составления расписаний.

Учебный или иной рабочий процесс невозможно представить без чётких временных границ, в рамках которых он происходит. В настоящее время в большинстве учебных заведений расписание составляется или редактируется вручную, чтобы избежать данной рутинной работы и было принято решение разработать приложения для генерации готовых расписаний. Данная разработка позволит сэкономить определённое время на составление самого расписания и более объективно распределить нагрузку.

Анализ предметной области. Декомпозиция задач

В первую очередь для формирования списка задач необходимо проанализировать модели, с которыми система будет работать и основные требования к функционалу системы. Модели системы представлены ниже на таблице 1.

Таблица 1. Модели системы

Название модели	Описание
Класс	Сущность класса учеников, необходима для описания групп учеников, для которых генерируется расписание.
Преподаватель	Сущность преподавателя, используется для описания существующих преподавателей, для распределения учеников между ними.
Предмет	Сущность предмета, используется для связывания соответствующих классов и преподавателей.
Аудитория	Сущность аудиторий, необходима для понимания, в каком месте будет проводиться соответствующий урок.
Урок	Сущность урока, является сущностью, отражающей объекты класса, преподавателя, предмета и аудитории, используемые для проведения занятия.
Расписание	Сущность расписания, используется для представления итогового расписания, содержит в себе сущности уроков.

Требования к функционалу системы:

1. Взаимодействие с классами
2. Взаимодействие с преподавателями
3. Взаимодействие с предметами
4. Взаимодействие с аудиториями

5. Взаимодействие с уроками

6. Генерация расписания

После анализа предметной области можно приступить к декомпозиции задач. Результат декомпозиции представлен ниже в таблице 2.

Таблица 2. Декомпозиция задач

Эпик	Задачи	Результат	Критерий готовности
Этап проектирования			
Создание исходного проекта		Создан backend модуль и подключены необходимые фреймворки для работы с базой данных	Модули сконфигурированы и готовы к запуску, backend способен взаимодействовать с базой данных
Реализация функционала предмета			
	Конструирование сущности предмета	Написан класс сущности предмета, описаны поля класса, у каждого поля верный тип.	В программном коде добавлен новый класс предмета, в данном классе содержатся необходимые для анализа поля, поля могут быть корректно заполнены.
	Реализация экранов предмета	Реализованы экран отображения данных о предметах и экран добавления/редактирования новых/существующих предметов	На первом экране, при существующих данных, отображён список предметов. При помощи второго экрана можно успешно создавать/редактировать предметы
	Реализация CRUD операций	Реализован механизм чтения/создания/изменения/удаления предмета	При использовании соответствующих методов можно изменять содержимое списка

			предметов
Реализация функционала аудитории			
	Конструирование сущности аудитории	Написан класс сущности аудитории, описаны поля класса, у каждого поля верный тип.	В программном коде добавлен новый класс аудитории, в данном классе содержатся необходимые для анализа поля, поля могут быть корректно заполнены.
	Реализация экранов аудитории	Реализованы экран отображения данных о аудиториях и экран добавления/редактирования новых/существующих аудиторий	На первом экране, при существующих данных, отображён список аудиторий. При помощи второго экрана можно успешно создавать/редактировать аудитории
	Реализация CRUD операций	Реализован механизм чтения/создания/изменения/удаления аудиторий	При использовании соответствующих методов можно изменять содержимое списка аудиторий
Реализация функционала класса			
	Конструирование сущности класса	Написан класс сущности класса, описаны поля класса, у каждого поля верный тип, настроены необходимые связи.	В программном коде добавлен новый класс аудитории, в данном классе содержатся необходимые для анализа поля, поля могут быть корректно заполнены.
	Реализация экранов класса	Реализованы экран отображения данных о аудиториях и экран добавления/редактирования новых/существующих классов	На первом экране, при существующих данных, отображён список классов. При помощи второго экрана можно успешно создавать/редактиро

			вать классы
	Реализация CRUD операций	Реализован механизм чтения/создания/изменения/удаления класса	При использовании соответствующих методов можно изменять содержимое списка классов
Реализация функционала учителя			
	Конструирование сущности учителя	Написан класс сущности учителя, описаны поля класса, у каждого поля верный тип, настроены необходимые связи.	В программном коде добавлен новый класс учителя, в данном классе содержатся необходимые для анализа поля, поля могут быть корректно заполнены.
	Реализация экранов учителя	Реализованы экран отображения данных о учителе и экран добавления/редактирования новых/существующих учителей	На первом экране, при существующих данных, отображён список учителей. При помощи второго экрана можно успешно создавать/редактировать объекты учителей
	Реализация CRUD операций	Реализован механизм чтения/создания/изменения/удаления учителей	При использовании соответствующих методов можно изменять содержимое списка учителей
Реализация функционала урока			
	Конструирование сущности урока	Написан класс сущности урока, описаны поля урока, у каждого поля верный тип.	В программном коде добавлен новый класс урока, в данном классе содержатся необходимые для анализа поля, поля могут быть корректно

			заполнены.
	Реализация экранов уроков	Реализованы экран отображения данных о уроке и экран добавления/редактирования новых/существующих уроков	На первом экране, при существующих данных, отображён список уроков. При помощи второго экрана можно успешно создавать/редактировать объекты уроков
	Реализация CRUD операций	Реализован механизм чтения/создания/изменения/удаления уроков	При использовании соответствующих методов можно изменять содержимое списка уроков
Реализация функционала расписания			
	Конструирование сущности расписания	Написан класс сущности урока, описаны поля урока, у каждого поля верный тип, настроены необходимые связи.	В программном коде добавлен новый класс расписания, в данном классе содержатся поля, отражающие все необходимые параметры сущности.
	Реализация экрана расписания	Реализован экран отображения данных о расписании и переформирования расписания, присутствует возможность изменять расписание вручную	Отображаются данные о составленном расписании, при его изменении на UI, происходят соответствующие изменения на бэке
	Реализация CRUD операций	Реализован механизм чтения/создания/изменения/удаления уроков	При использовании соответствующих методов можно изменять содержимое списка уроков

	Реализация функции, составляющего расписание	При полученных входных данных алгоритм выдаёт адекватную оценку составленному расписанию. В конечном итоге выбирает лучшее расписание по оценке и предоставляет пользователю	Алгоритм позволяет получить на выходе составленное расписание, примерно учитывающее значения входных данных
--	--	--	---

Оценка объема работ по задачам

Для оценивая задач был выбран способ интервальной оценки согласно рекомендации из требований, а также способ был выбран благодаря возможности заранее просчитать и заложить риски в оценку. Результат оценки представлен ниже на таблице 3.

Таблица 3. Оценка задач

Название задачи	Минимальная оценка	Максимальная оценка
Создание исходного проекта	1	3
Реализация функционала предмета		
Конструирование сущности предмета	1	2
Реализация экранов предмета	5	8
Реализация CRUD операций	1	2
Реализация функционала аудитории		
Конструирование сущности аудитории	0,25	1
Реализация экранов аудитории	1	3
Реализация CRUD операций	0,5	1
Реализация функционала класса		
Конструирование сущности класса	1	3
Реализация экранов класса	1	3
Реализация CRUD операций	3	4
Реализация функционала учителя		
Конструирование сущности учителя	1	3
Реализация экранов учителя	4	5
Реализация CRUD операций	5	8
Реализация функционала урока		
Конструирование сущности урока	1	2
Реализация экранов уроков	2	4
Реализация CRUD операций	3	4
Реализация функционала расписания		
Конструирование сущности расписания	2	4

Реализация экрана расписания	4	6
Реализация CRUD операций	3	4
Реализация функции, составляющей расписание	8	12

К указанной оценке необходимо добавить оценку на этапы разработки, выделенные при планировании.

- Проектирование и анализ: 8 – 10 часов
- Тестирование: 4 – 6 часов

Итоговая оценка, с учетом оценки всех задач и этапов, составляет:

- Оценка **минимально** возможного времени – 59,75 часов
- Оценка **максимально** возможного времени – 99 часов

Планирование разработки

В качестве модели жизненного цикла была выбрана методология Kanban, что позволяет сформировать бэклог задач и изменять их статусы на Kanban доске, чтобы отчетливо видеть текущий статус разработки. Также данная методология является гибкой, что позволяет в любой момент внести новые требования для получения наилучшего конечного результата.

В качестве архитектуры системы была выбрана монолитная архитектура, так как весь функционал системы без трудностей можно реализовать без разделения на микросервисы и в приоритете стоит скорость разработки.

При проектировании в каждом приложении будут выделены определенные слои, имеющие свою область ответственности, что позволит наладить четкую коммуникацию между слоями и разграничить логику приложения от других аспектов.

После задачи были отсортированы по приоритету

1. Создание исходного проекта.
2. Конструирование сущности преподавателя.
2. Конструирование сущности класса.
2. Конструирование сущности предмета.
2. Конструирование сущности аудитории.
3. Реализация CRUD операций преподавателя.
3. Реализация CRUD операций класса.
3. Реализация CRUD операций предмета.
3. Реализация CRUD операций аудитории.
4. Конструирование сущности урока.
4. Конструирование сущности расписания.
4. Реализация функции, составляющей расписание.
5. Реализация CRUD операций урока.

5. Реализация CRUD операций расписания.
6. Реализация экрана преподавателя.
6. Реализация экрана класса.
6. Реализация экрана предмета.
6. Реализация экрана аудитории.
6. Реализация экрана урока.
6. Реализация экрана расписания.

После приоритезации были выделены этапы разработки

1. Проектирование и анализ
2. Конструирование MVP
3. Доработка основного функционала
4. Реализация дополнительных функций
5. Тестирование

Далее сформируем календарный график работ по этапам и укажем крайний срок для каждого этапа разработки. Результат представлен на таблице 4.

Таблица 4. Календарный график этапов

Название этапа	Конечный срок
Проектирование и анализ	24.11.2022
Конструирование MVP	28.11.2022
Доработка основного функционала	05.12.2022
Реализация дополнительных функций	08.12.2022
Тестирование	12.12.2022

Проектирование в конструировании

Use Case диаграмма

Для лучшего понимания какие задачи мы пытаемся решить при помощи создаваемого проекта и для понимания какие функции будут доступны пользователю нам необходимо рассмотреть use-case диаграмму. Данная последовательность проектирования позволит посмотреть на систему сверху вниз, в данном случае мы последовательно понижаем уровень абстракции с пользовательских сценариев, до уровня кода. На рисунке ниже представлены основные варианты использования приложения. Use Case диаграмма продемонстрирована на рисунке 1.

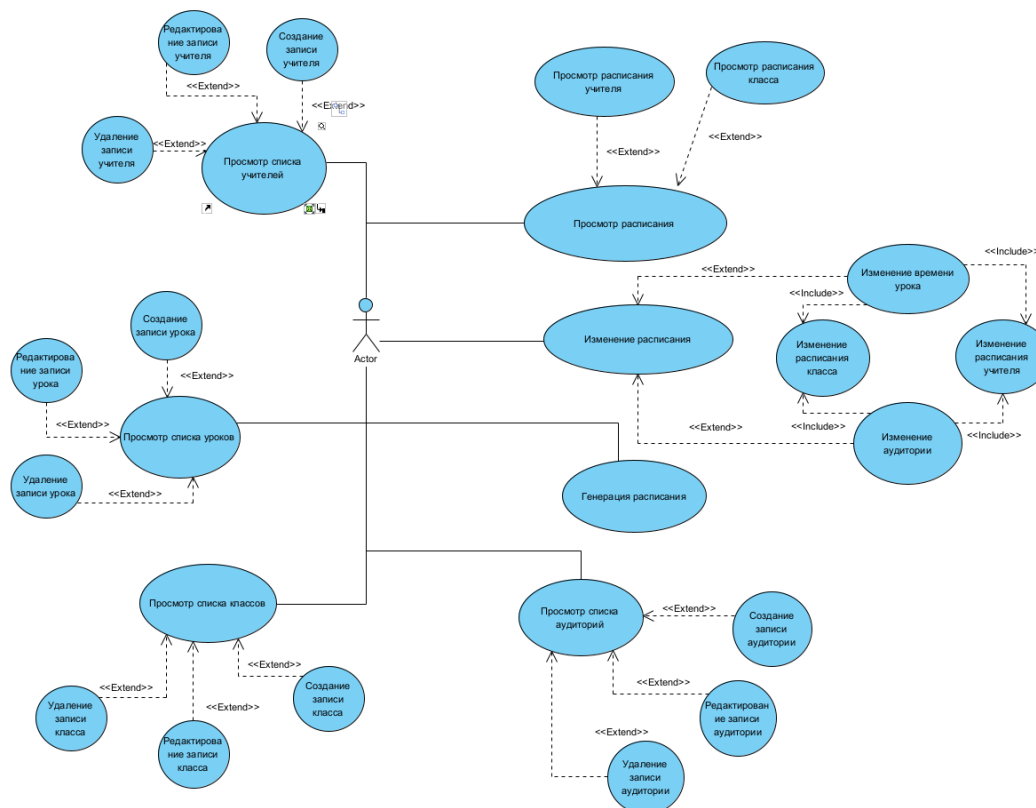


Рисунок 1. Use Case диаграмма авторизованного пользователя

ER диаграмма

Далее предлагаю рассмотреть основные сущности приложения, воспользовавшись ER диаграммой на рисунке ниже. Данная диаграмма

позволяет лучше ознакомиться с данными выбранной предметной области, начать обдумывание реализации обработки и взаимодействия объектов.

Class – таблица, представляющая собой сущность класса учеников и содержащая такие поля как

- id – уникальный идентификатор типа integer
- name – уникальное представление названия класса типа varchar (255)
- course – номер курса/класса группы учеников типа integer
- number_students – количество студентов в группе типа integer
- estimate – коэффициент класса, является субъективной оценкой, используется для смещения при генерации расписания в пользу определённого объекта, тип integer

Lesson – таблица, представляющая собой сущность урока и содержащая такие поля как

- id – уникальный идентификатор типа integer
- time – время отражающее время проведения урока типа timestamp
- type – тип урока, параметр отражающий вид урока (лекция/практика) типа boolean

Teacher – таблица, представляющая собой сущность преподавателя и содержащая такие поля как

- id – уникальный идентификатор типа integer
- name – уникальное представление имени преподавателя типа varchar (255)
- priority – коэффициент «важности» преподавателя, используется для смещения при генерации расписания в пользу определённого объекта, тип integer
- start_day – время начала занятий преподавателя типа timestamp

- end_day – время завершения занятий преподавателя типа timestamp

Classroom – таблица, представляющая собой сущность аудитории и содержащая такие поля как:

- id – уникальный идентификатор типа integer
- name – уникальное представление названия аудитории типа varchar (255)
- capacity – представление вместимости аудитории типа integer
- type – представление типа аудитории (лекционная/для практик) тип Boolean

Subject – таблица, представляющая собой сущность предмета и содержащая такие поля как:

- id – уникальный идентификатор типа integer
- name – представление названия предмета типа varchar (255)

Subject_Class – таблица, представляющая связь многие ко многим между группой студентов и предметом.

Teacher_Subject – таблица, представляющая связь многие ко многим между преподавателем и предметом.

Teacher_Week – таблица, представляющая связь многие ко многим между преподавателем и рабочим днём недели.

Результат анализа представлен в виде ER диаграммы, продемонстрированной на рисунке 2.

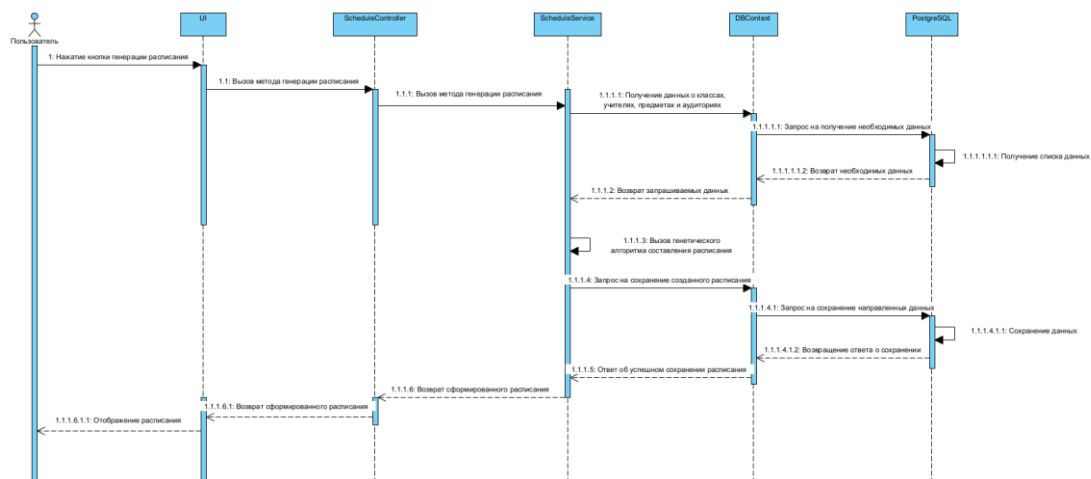


Рисунок 3. Sequence диаграмма составления расписания

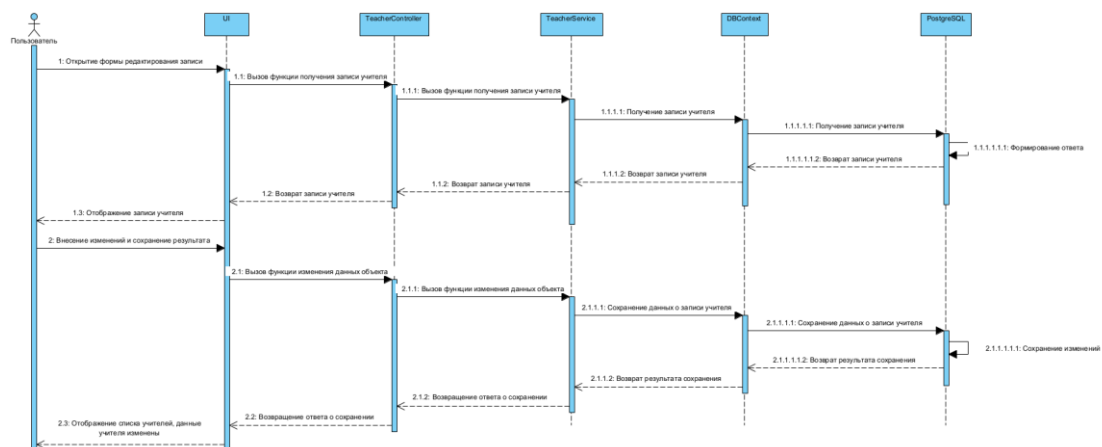


Рисунок 4. Sequence диаграмма редактирования объекта

Обоснование выбора процедур процесса конструирования

Язык и платформа разработки

В качестве платформы для серверной части была выбрана платформа .Net, данная платформа предоставляет необходимый функционал для решения поставленных задач. Также имеет готовые библиотеки, использование которых производится путём установки соответствующих Nuget пакетов.

Инструменты

- VisualParadigm – инструмент для создания диаграмм
- Visual Studio - IDE для разработки, рефакторинга, и генерации кода приложения.

Технологии

«Asp.Net.Core» – фреймворк для создания Web-приложений.

«EntityFrameworkCore» – фреймворк для работы с БД.

«MSSQL» – база данных, используемая в приложении.

«NUnit» - фреймворк для тестирования приложения.

Техника кодирования

В процессе разработки системы будут соблюдены определенные соглашения о стиле кода:

- Название функций и полей происходит в формате camelCase
- Константы называются заглавными буквами.
- Вокруг операторов (= + - * /) и после запятых ставятся пробелы
- Для отступов блоков кода используются 4 пробела.
- Общие правила определения объектов:
 - а. Открывающая скобка отделяется от названия объекта пробелом
 - б. Закрывающая скобка помещается на новую строку, без начальных пробелов.

В процессе разработки использовались техники по организации исходного кода, такие как:

- Разделение на слои – слой API, слой логики, слой взаимодействия с БД
- В названии объекта последним словом фиксируется, к какому слою он относится
- В каждом слое группировка компонентов по назначению компонента -
Слой API (контроллеры, модели, сервисы, исключения, расширения),
слой доступа к данным (модели, миграции), слой тестирования
(модульные тесты, интеграционные тесты)

Прототип приложения

Для формирования прототипа были выделены следующие задачи

- Создание исходного проекта
- Создание моделей данных
- Реализация интеграции с БД
- Реализация CRUD операций для сущностей системы
- Реализация логики генерации расписания
- Реализация экранных форм приложения

В ходе выполнения работ были реализованы соответствующие сущности системы, реализована интеграция с БД, настроено взаимодействие между объектами данных, необходимыми для генерации расписания и непосредственно сам генетический алгоритм для его создания.

Этапы генерации расписания представлены на рисунке 5.

```
CourseWork.exe" (CoreCLR: clrhost). Загружено "C:\Users\Владислав\Desktop\Учб66\ScheduleWork\CourseWork\bin\Debug\net6.0-windows\Microsoft.SqlServer.Server.dll". Загрузка сим...
Начальное распределение завершено
"CourseWork.exe" (CoreCLR: clrhost). Загружено "C:\Program Files\dotnet\shared\Microsoft.NETCore.App\6.0.0\System.Diagnostics.Process.dll". Загрузка символов пропущена. Модуль
"CourseWork.exe" (CoreCLR: clrhost). Загружено "C:\Program Files\dotnet\shared\Microsoft.NETCore.App\6.0.0\System.Diagnostics.Process.dll". Загрузка символов пропущена. Модуль оптимизирован,
Количество особей перед отсевом 25
[7, 198,87714047999998]
[2, 198,6758944]
[11, 193,35862156799998]
Итерация 1 завершена
Количество особей после отсева 3
Поток 0x2b14 завершился с кодом 0 (0x0).
Количество особей перед отсевом 75
[27, 199,71615423999998]
[0, 199,70124927999998]
[67, 192,933623808]
Итерация 2 завершена
Количество особей после отсева 3
Поток 0x3890 завершился с кодом 0 (0x0).
Поток 0x5668 завершился с кодом 0 (0x0).
Поток 0x3300 завершился с кодом 0 (0x0).
Поток 0x135c завершился с кодом 0 (0x0).
Поток 0x3e28 завершился с кодом 0 (0x0).
Поток 0x4c38 завершился с кодом 0 (0x0).
Поток 0x5148 завершился с кодом 0 (0x0).
Поток 0x10c0 завершился с кодом 0 (0x0).
Количество особей перед отсевом 75
[72, 200,27207424]
[9, 200,26615424]
[14, 192,886727168]
Итерация 3 завершена
Количество особей после отсева 3
Количество особей перед отсевом 75
[67, 201,65703424]
[12, 201,18775424]
[13, 192,84249292799998]
Итерация 4 завершена
Количество особей после отсева 3
Количество особей перед отсевом 75
[34, 201,31911423999998]
[9, 201,18775424]
[56, 194,48354892799998]
Итерация 5 завершена
Количество особей после отсева 3
Генерация завершена
Поток 0x4580 завершился с кодом 0 (0x0).
```

Рисунок 5. Этапы генерации расписания

Пример сгенерированного расписания (сущность урок в БД) представлен на рисунке 6 (выборка отсортирована по учителю). По данным

можно определить что алгоритм действительно отработал и определённые предметы были изменены в своём порядке, например, у teacher с id = 3. Также сохранена логика распределения, одному классу не достались разные учителя в рамках одного предмета. Но само изменение данных не значительно, это можно увидеть распределению предметов, это обусловлено характером тестовых данных, малым количеством учителей и малым количеством классов, в отличие от количества предметов (оно примерно равно количеству учителей и классов).

time	classID	teacherID	classroomID	subjectID
2	1	1	4	1
3	1	1	4	1
4	1	1	4	1
5	2	1	1	1
6	2	1	1	1
10	2	1	1	1
11	3	1	4	1
12	3	1	4	1
13	3	1	4	1
14	4	1	7	1
18	4	1	7	1
19	4	1	7	1
20	2	1	1	2
21	2	1	1	2
22	2	1	1	2
34	3	1	4	2
35	3	1	4	2
36	3	1	4	2
1	3	2	4	14
2	3	2	6	14
3	3	2	6	14
4	7	2	7	14
5	7	2	7	14
9	7	2	7	14
10	2	2	2	13
11	2	2	1	13
12	6	2	1	12
13	6	2	1	12
17	6	2	1	12
18	7	2	8	12
19	7	2	8	12
20	7	2	7	12
21	4	2	7	11
25	4	2	7	11
26	6	2	1	11
27	6	2	1	11
1	1	3	6	7
2	1	3	1	7
3	2	3	1	7
9	2	3	1	7
10	3	3	4	6
11	3	3	6	6
17	5	3	7	6
18	5	3	4	6
19	7	3	4	6
25	7	3	8	6
26	1	3	4	9
27	1	3	4	9
33	5	3	7	9
34	5	3	7	9
35	6	3	1	10
41	6	3	1	10
42	8	3	1	10
43	8	3	1	10
44	8	4	7	6

Рисунок 6. Пример сгенерированного расписания

Также реализовано дополнительное смещение данных, когда предметы могут быть отданы разным учителям в рамках одного класса. Пример данного расписания представлен на рисунке 7.

time	classID	teacherID	classroomID	subjectID
2	1	1	4	1
4	1	1	4	1
5	2	1	1	1
6	2	1	1	1
10	2	1	1	1
11	3	1	4	1
12	2	1	1	6
13	3	1	4	1
14	4	1	7	1
19	4	1	7	1
20	2	1	1	2
21	2	1	1	2
22	2	1	1	2
33	3	1	4	2
34	3	1	4	2
35	3	1	4	2
36	3	1	4	1
37	6	1	1	1
38	4	1	7	1
2	3	2	6	14
3	3	2	4	14
4	3	2	6	14
5	7	2	7	14
9	7	2	7	14
10	2	2	2	13
11	2	2	1	13
12	6	2	2	12
13	6	2	1	12
17	6	2	1	12
19	7	2	8	12
20	7	2	7	12
21	4	2	7	11
25	7	2	7	14
26	7	2	7	12
27	6	2	1	11
36	6	2	1	11
37	4	2	7	11
1	1	3	4	7
2	1	3	1	7
3	5	3	7	7
9	2	3	1	7
10	7	3	7	6
11	3	3	6	6
17	5	3	7	6
18	6	3	1	6
19	3	3	4	6
25	7	3	8	6
26	1	3	4	9
27	1	3	4	9
33	8	3	1	9
34	5	3	7	9
35	6	3	1	10
41	5	3	7	10
42	2	3	1	10
43	8	3	1	10

Рисунок 7. Пример сгенерированного расписания со смешением классов учителей внутри класса

В ходе выполнения прототипа были уточнены функциональные требования и реализован функционал генетического алгоритма составления расписания. По итогам выполнения прототипа можно сделать вывод, что задачи были оценены некорректно, указанные временные значения слишком малы для решения поставленной задачи с учётом текущих навыков.

Итоги разработки

В ходе выполнения работы возникли трудности, связанные с реализацией генетического алгоритма, основного функционала программы.

Первая проблема была связана с неудобством использования коллекций, отчего время выполнения программы сильно увеличивалось. Вместе с этим появилась вторая проблема, что изначально выбранная структура данных требовала изменений, но поскольку об этом стало известно лишь на конечном этапе, во время прохождения процесса генерации, то потребовались значительные изменения функционала.

Вторая проблема была связана с распределением ответственности между функций оценки и мутацией, поскольку изначально было принято решение производить потомков с ограничением, что на один класс, в рамках одного предмета, может быть лишь один учитель, то логика генерации новых поколений заметно усложнилась.

Третья проблема была связана с решением производить мутации не по одной сущности (преподаватель или класс), а сразу по двум, то большое число времени ушло на сохранение согласованности данных.

Четвёртая проблема была связана с отсутствием корректных аналогичных решений для данных ограничений (третья проблема) в общем доступе, отчего потребовалось дополнительное время на разработку собственного.

Для удержания разработки проекта в установленных временных рамках было принято решение не реализовывать GUI функционал, на данный момент реализована лишь back-end составляющая, с взаимодействием с БД и генерацией самого расписания.

В результате, в указанный изначально срок была реализована лишь часть функционала, а именно back-end составляющая. Оставшиеся 19 часов

можно было бы потратить на реализацию UI части, но за указанный промежуток времени данный функционал полностью реализовать бы не удалось, но поскольку несвязанный частично реализованный UI не имеет смысла для пользователя, данный блок не был реализован.

Оценка задач с фактическим временем на реализацию представлена ниже на таблице 5.

Таблица 5. Фактическое время на реализацию

Название задачи	Минимальная оценка	Максимальная оценка	Затраченное время
Проектирование	8	10	8
Создание исходного проекта	1	3	2
Реализация функционала предмета			
Конструирование сущности предмета	1	2	1
Реализация экранов предмета	5	8	-
Реализация CRUD операций	1	2	2
Реализация функционала аудитории			
Конструирование сущности аудитории	0,25	1	1
Реализация экранов аудитории	1	3	-
Реализация CRUD операций	0,5	1	1
Реализация функционала класса			
Конструирование сущности класса	1	3	1
Реализация экранов класса	1	3	-
Реализация CRUD операций	3	4	3
Реализация функционала учителя			
Конструирование сущности учителя	1	3	2
Реализация экранов учителя	4	5	-
Реализация CRUD операций	5	8	5

Реализация функционала уроков			
Конструирование сущности уроков	1	3	1
Реализация экранов уроков	2	4	-
Реализация CRUD операций	3	4	4
Реализация функционала расписания			
Конструирование сущности расписания	2	4	4
Реализация экрана расписания	4	6	-
Реализация CRUD операций	3	4	-
Реализация генетического алгоритма генерации расписания	8	12	40
Тестирование	4	6	8
Итого	59,75	99	83

В ходе выполнения курсовой работы были закреплены на практике навыки написания серверной части приложений на языке C#, оценивания временной сложности алгоритмов и знания элементарной математики.

Время исполнения урезанной версии программы соответствует оценке, время исполнения полного числа изначальных требований значительно бы превысило максимальную оценку.

Тестирование в конструировании

Для определения видов тестирования нам необходимо понимать, что у данного проекта отсутствует GUI составляющая и приложение является монолитным и десктопным, в соответствии с этим отсекается определённая часть возможных тестов.

Также основой приложения выступает генетический алгоритм, основанный на «случайных» изменениях особей расписания. При таком подходе можно лишь проверить результат определённого этапа алгоритма и его завершения, логику самого алгоритма проверять нет необходимости.

Тем самым принято решение использовать интеграционные тесты для проверки взаимодействия слоя логики и слоя хранилища данных системы и e2e тесты и анализ полученного расписания. Сверка результатов тестов будет производиться вручную.

Для лучшего тестового покрытия будут добавлены модульные тесты, покрывающие логику использования сущностей, проверки аналогичны интеграционным, но вместо взаимодействия с БД будет использован Mock.

Тест сьюты для тестирования представлены в таблице 6.

Таблица 6. Тест сьюты тестирования

ID	Автор	Приоритет	Заголовок	Список тест кейсов
1	Мазяков В.В.	1	CRUD класса (БД)	1.1 Добавление нового класса 1.2 Добавление нового класса без имени (обязательный параметр) 1.3 Добавление нового класса без оценки (необязательный параметр) 1.4 Добавление нового класса с существующим именем 1.5 Добавление предметов

				существующему классу 1.6 Редактирование существующего класса 1.7 Просмотр конкретного класса 1.8 Просмотр списка классов 1.9 Просмотр пустого списка классов 1.10 Удаление существующего класса 1.11 Удаление несуществующего класса
2	Мазяков В.В.	1	CRUD аудитории (БД)	2.1 Добавление новой аудитории 2.2 Добавление новой аудитории без вместимости 2.3 Добавление новой аудитории с существующим именем 2.4 Редактирование существующей аудитории 2.5 Просмотр конкретной аудитории 2.6 Просмотр списка аудиторий 2.7 Удаление существующей аудитории 2.8 Удаление несуществующей аудитории
3	Мазяков В.В.	1	CRUD учителя (БД)	3.1 Добавление нового учителя 3.2 Добавление нового учителя без имени (обязательный параметр) 3.3 Добавление нового учителя без приоритета (необязательный параметр) 3.4 Добавление нового учителя с существующим именем 3.5 Добавление связей к объекту учителя 3.6 Редактирование существующего учителя 3.7 Просмотр конкретного учителя 3.8 Просмотр списка учителей 3.9 Удаление существующего учителя

				3.10 Удаление несуществующего учителя
4	Мазяков В.В.	1	CRUD предмета (БД)	4.1 Добавление нового предмета 4.2 Добавление нового предмета без имени (обязательный параметр) 4.3 Добавление нового предмета с существующим именем 4.4 Редактирование существующего предмета 4.5 Просмотр конкретного предмета 4.6 Просмотр списка предметов 4.7 Удаление существующего предмета 4.8 Удаление несуществующего предмета

Тест кейсы для интеграционного тестирования представлены на таблице 7.

Таблица 7. Тест кейсы для интеграционного тестирования

ID	Название	Тип	Предусловие	Шаги	Ожидаемый результат
1.1	Добавление нового класса	Позитивный	-	1. Создать запрос на добавление записи класса (тест, 1, 1, 1, 1) 2. Выполнить запрос на добавление	1. В БД добавлена запись таблицы класса с данными (тест, 1, 1, 1, 1)
1.2	Добавление нового класса без имени	Негативный	-	1. Создать запрос на добавление записи класса (null, 1, 1, 1, 1) 2. Выполнить запрос на добавление	1. БД возвращает системе ошибку, данное поле является обязательным
1.3	Добавление нового класса без оценки	Позитивный	-	1. Создать запрос на добавление записи класса (тест, 1, 1, 1, null)	1. В БД добавлена запись таблицы класса с данными (тест, 1, 1, 1, null)

				2. Выполнить запрос на добавление	
1.4	Добавление нового класса с существующим именем	Негативный	В БД содержится запись класса с name = тест	1. Создать запрос на добавление записи класса (тест, 1, 1, 1, null) 2. Выполнить запрос на добавление	1. БД возвращает системе ошибку, поле имени должно быть уникальным
1.5	Добавление предметов существующему классу	Позитивный	В БД содержится минимум одна запись класса и одна запись предмета	1. Создать и выполнить запрос на связывание существующих класса и предмета	1. В БД добавлена запись в таблице ClassSubjects, содержащая ID связываемых элементов
1.6	Редактирование существующего класса	Позитивный	В БД содержится минимум одна запись класса, известно ID данного элемента, данные записи отличаются от указанных в шагах	1. Создать запрос на редактирование записи класса (тест2, 1, 1, 1, 1) 2. Выполнить запрос на редактирование	1. В БД запись для редактирования имеет изменённые данные (тест2, 1, 1, 1, 1)
1.7	Просмотр конкретного класса	Позитивный	В БД содержится минимум одна запись класса, известно ID данного элемента	1. Создать запрос на получение записи класса с ID из предусловия. 2. Выполнить запрос на получение выборки	1. Система получает соответствующие данные из БД
1.8	Просмотр списка классов	Позитивный	В БД содержится минимум одна запись класса	1. Создать запрос на получение всех записей классов 2. Выполнить запрос на получение выборки	1. Система получается соответствующие данные из БД

1.9	Просмотр пустого списка классов	Позитивный	В таблице Class не содержится ни одной записи	1. Создать запрос на получение всех записей класса 2. Выполнить запрос на получение выборки	1. Система получает пустую выборку
1.10	Удаление существующего класса	Позитивный	В БД содержится минимум одна запись класса, известно ID данного элемента	1. Создать запрос на удаление класса с указанным ID 2. Выполнить запрос на удаление	1. В БД отсутствует запись с указанным ID
1.11	Удаление несуществующего класса	Негативный	Известно ID отсутствующего элемента в БД	1. Создать запрос на удаление класса с указанным ID 2. Выполнить запрос на удаление	1. Данный запрос возвращает внутреннюю ошибку, запрос не доходит до БД
2.1	Добавление новой аудитории	Позитивный	-	1. Создать запрос на добавление записи аудитории (тест, 1, true) 2. Выполнить запрос на добавление	1. В БД добавлена запись таблицы аудитории с данными (тест, 1, true)
2.2	Добавление новой аудитории без параметра вместимости	Негативный	-	1. Создать запрос на добавление записи аудитории (тест, 1, null) 2. Выполнить запрос на добавление	1. БД возвращает системе ошибку, данное поле является обязательным
2.3	Добавление новой аудитории с существующим именем	Негативный	В БД содержится запись аудитории с name = тест	1. Создать запрос на добавление записи аудитории (тест, 1, 1) 2. Выполнить запрос на	1. БД возвращает системе ошибку, поле имени должно быть уникальным

				добавление	
2.4	Редактирование существующей аудитории	Позитивный	В БД содержится запись аудитории с name = тест	1. Создать запрос на редактирование записи аудитории (тест2, 1, 1) 2. Выполнить запрос на редактирование	1. В БД запись для редактирования имеет изменённые данные (тест2, 1, 1)
2.5	Просмотр конкретной аудитории	Позитивный	В БД содержится минимум одна запись аудитории, известно ID данного элемента	1. Создать запрос на получение записи аудитории с ID из предусловия 2. Выполнить запрос на получение выборки	1. Система получает соответствующие данные из БД
2.6	Просмотр списка аудиторий	Позитивный	В БД содержится минимум одна запись аудитории	1. Создать запрос на получение всех записей аудиторий	1. Система получается соответствующие данные из БД
2.7	Удаление существующей аудитории	Позитивный	В БД содержится минимум одна запись аудитории, известно ID данного элемента	1. Создать запрос на удаление аудитории с указанным ID 2. Выполнить запрос на удаление	1. В БД отсутствует запись с указанным ID
2.8	Удаление несуществующей аудитории	Негативный	Известно ID отсутствующего элемента в БД	1. Создать запрос на удаление аудитории с указанным ID 2. Выполнить запрос на удаление	1. Данный запрос возвращает внутреннюю ошибку, запрос не доходит до БД
3.1	Добавление нового учителя	Позитивный	-	1. Создать запрос на добавление записи учителя (тест, 1, 1, 2, 2) 2. Выполнить	1. В БД добавлена запись таблицы учителя с данными (тест, 1, 1, 2, 2)

				запрос на добавление	
3.2	Добавление нового учителя без имени	Негативный	-	1. Создать запрос на добавление записи учителя (null, 1, 1, 2, 2) 2. Выполнить запрос на добавление	1. БД возвращает системе ошибку, данное поле является обязательным
3.3	Добавление нового учителя без приоритета	Позитивный	-	1. Создать запрос на добавление записи учителя (тест, null, 1, 2, 2) 2. Выполнить запрос на добавление	1. В БД добавлена запись таблицы учителя с данными (тест, null, 1, 2, 2)
3.4	Добавление нового учителя с существующим именем	Негативный	В БД содержится запись учителя с name = тест	1. Создать запрос на добавление записи учителя (тест, 1, 1, 2, 2) 2. Выполнить запрос на добавление	1. БД возвращает системе ошибку, поле имени должно быть уникальным
3.5	Добавление связей к объекту учителя	Позитивный	В БД содержится запись учителя с name = тест	1. Создать и выполнить запрос на связывание существующих учителя + предмета	1. В БД добавлены записи в таблице TeacherSubjects, содержащая ID связываемых элементов
3.6	Редактирование существующего учителя	Позитивный	В БД содержится минимум одна запись учителя, известно ID данного элемента, данные записи отличаются от указанных в шагах	1. Создать запрос на редактирование записи учителя (тест2, 1, 1, 2, 2) 2. Выполнить запрос на редактирование	1. В БД запись для редактирования имеет изменённые данные (тест2, 1, 1, 2, 2)
3.7	Просмотр	Позитивный	В БД содержится	1. Создать запрос на получение	1. Система

	конкретного учителя		минимум одна запись учителя, известно ID данного элемента	записи учителя с ID из предусловия. 2. Выполнить запрос на получение выборки	получает соответствующие данные из БД
3.8	Просмотр списка учителей	Позитивный	В БД содержится минимум одна запись учителя	1. Создать запрос на получение всех записей учителей 2. Выполнить запрос на получение выборки	1. Система получается соответствующие данные из БД
3.9	Удаление существующего учителя	Позитивный	В БД содержится минимум одна запись учителя, известно ID данного элемента	1. Создать запрос на удаление учителя с указанным ID 2. Выполнить запрос на удаление	1. В БД отсутствует запись с указанным ID
3.10	Удаление несуществующего учителя	Негативный	Известно ID отсутствующего элемента в БД	1. Создать запрос на удаление учителя с указанным ID 2. Выполнить запрос на удаление	1. Данный запрос возвращает внутреннюю ошибку, запрос не доходит до БД
4.1	Добавление нового предмета	Позитивный	-	1. Создать запрос на добавление записи предмета (тест, 1)	1. В БД добавлена запись таблицы предмета с данными (тест, 1)
4.2	Добавление нового предмета без имени	Негативный	-	1. Создать запрос на добавление записи предмета (null, 1)	1. БД возвращает системе ошибку, данное поле является обязательным
4.3	Добавление нового предмета с	Негативный	В БД содержится запись предмета с	1. Создать запрос на добавление	1. БД возвращает системе ошибку,

	существующим именем		name = тест	записи предмета (тест, 1)	поле имени должно быть уникальным
4.4	Редактирование существующего предмета	Позитивный	В БД содержится минимум одна запись предмета, известно ID данного элемента, данные записи отличаются от указанных в шагах	1. Создать запрос на редактирование записи предмета (тест2, 1) 2. Выполнить запрос на редактирование	1. В БД запись для редактирования имеет изменённые данные (тест2, 1)
4.5	Просмотр конкретного предмета	Позитивный	В БД содержится минимум одна запись предмета, известно ID данного элемента	1. Создать запрос на получение записи предмета с ID из предусловия. 2. Выполнить запрос на получение выборки	1. Система получает соответствующие данные из БД
4.6	Просмотр списка предметов	Позитивный	В БД содержится минимум одна запись предмета	1. Создать запрос на получение всех записей предмета 2. Выполнить запрос на получение выборки	1. Система получается соответствующие данные из БД
4.7	Удаление существующего предмета	Позитивный	В БД содержится минимум одна запись предмета, известно ID данного элемента	1. Создать запрос на удаление предмета с указанным ID 2. Выполнить запрос на удаление	1. В БД отсутствует запись с указанным ID
4.8	Удаление несуществующего предмета	Негативный	Известно ID отсутствующего элемента в БД	1. Создать запрос на удаление предмета с указанным ID 2. Выполнить	1. Данный запрос возвращает внутреннюю ошибку, запрос не доходит до БД

				запрос на удаление	
--	--	--	--	--------------------	--

Тест сьюты для e2e тестирования представлены на таблице 8.

Таблица 8. Тест сьюты для e2e тестирования генерации расписания

ID	Автор	Приоритет	Заголовок	Список тест кейсов
1	Мазяков В. В.	2	Проверка логики генерации расписания	5.1 Наличие результата генетического алгоритма 5.2 Наличие мутаций учителей 5.3 Наличие мутаций классов 5.4 Количество особей на этапе размножения 5.5 Число отбираемых после фитнес-функции особей 5.6 Корректность турнирного отбора 5.7 Согласованность данных учеников полученного расписания 5.8 Согласованность данных учителей полученного расписания 5.9 Корректность распределения аудиторий

Тест кейсы для e2e тестирования представлены на таблице 9.

Таблица 9. Тест кейсы для e2e тестирования генерации расписания

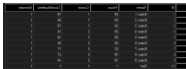
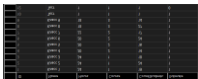
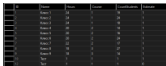
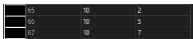
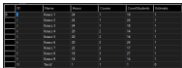
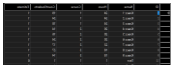
ID	Название	Тип	Предусловие	Шаги	Ожидаемый результат
5.1	Наличие результата генетического алгоритма	Позитивный	БД заполнена данными о учителях, классах, аудиториях и предметах, данные связаны друг с другом	1. Запустить приложение 2. Дождаться окончания работы программы	1. В БД добавлены соответствующие записи уроков

5.2	Наличие мутаций учителя	Позитивный	БД заполнена данными о учителях, классах, аудиториях и предметах, данные связаны друг с другом. Установлено 1 повторение мутаций и 1 количество различных мутаций.	1. Поставить точку останова перед мутацией объекта 2. Поставить точку останова после мутации объекта 3. Сверить изменения	1. Объект особи отличается после мутации.
5.3	Наличие мутаций класса	Позитивный	БД заполнена данными о учителях, классах, аудиториях и предметах, данные связаны друг с другом. Установлено 1 повторение мутаций и 1 количество различных мутаций.	1. Поставить точку останова перед мутацией объекта 2. Поставить точку останова после мутации объекта 3. Сверить изменения	1. Объект особи отличается после мутации.
5.4	Количество особей на этапе размножения	Позитивный	БД заполнена данными о учителях, классах, аудиториях и предметах, данные связаны друг с другом	1. Поставить точку останова перед этапом размножения 2. Поставить точку останова после этапа размножить	1. Количество особей равно числу особей до размножения * п.
5.5	Число отбираемых после фитнес-функции особей	Позитивный	БД заполнена данными о учителях, классах, аудиториях и предметах, данные связаны друг с	1. Поставить точку останова перед этапом отбора 2. Поставить точку останова после	1. Количество особей равно пороговому значению турнирного отбора (3)

			другом	этапа отбора	
5.6	Корректность турнирного отбора	Позитивный	БД заполнена данными о учителях, классах, аудиториях и предметах, данные связаны друг с другом	1. Поставить точку останова после определения оценок особей 2. Поставить точку останова после этапа отбора	1. Отбор пройден особями с наивысшими оценками
5.7	Согласованность данных учеников полученного расписания	Позитивный	БД заполнена данными о учителях, классах, аудиториях и предметах, данные связаны друг с другом	1. Запустить приложение 2. Дождаться окончания работы программы	1. У классов не может быть уроков по предметам, не входящих в их список.
5.8	Согласованность данных учителей полученного расписания	Позитивный	БД заполнена данными о учителях, классах, аудиториях и предметах, данные связаны друг с другом	1. Запустить приложение 2. Дождаться окончания работы программы	1. У учителей не может быть уроков по предметам, не входящих в их список.
5.9	Корректность распределения аудиторий	Позитивный	БД заполнена данными о учителях, классах, аудиториях и предметах, данные связаны друг с другом	1. Запустить приложение 2. Дождаться окончания работы программы	1. У классов во время уроков выданные аудитории по размеру больше или равны количеству учеников в классе

Отчет по тестированию

Таблица 10. Отчёт о выполнении тестирования

Добавление нового класса	успешно 
Добавление нового класса без имени (обязательный параметр)	успешно
Добавление нового класса без оценки (необязательный параметр)	успешно 
Добавление нового класса с существующим именем	провалено 
Добавление предметов существующему классу	успешно 
Редактирование существующего класса	успешно 
Просмотр конкретного класса	успешно
Просмотр списка классов	успешно
Просмотр пустого списка классов	успешно
Удаление существующего класса	успешно 
Удаление несуществующего класса	успешно
Добавление новой аудитории	успешно
Добавление новой аудитории без вместимости	успешно

Добавление новой аудитории с существующим именем	провалено
Редактирование существующей аудитории	успешно
Просмотр списка аудиторий	успешно
Удаление существующей аудитории	успешно
Удаление несуществующей аудитории	успешно
Добавление нового учителя	успешно
Добавление нового учителя без имени (обязательный параметр)	успешно
Добавление нового учителя без приоритета (необязательный параметр)	успешно
Добавление нового учителя с существующим именем	провалено
Добавление связей к объекту учителя	успешно
Редактирование существующего учителя	успешно
Просмотр конкретного учителя	успешно
Просмотр списка учителей	успешно
Удаление существующего учителя	успешно
Удаление несуществующего учителя	успешно
Добавление нового предмета	успешно
Добавление нового предмета без имени (обязательный параметр)	успешно
Добавление нового предмета с существующим именем	провалено
Просмотр конкретного предмета	успешно
Просмотр списка предметов	успешно
Удаление существующего предмета	успешно

Удаление несуществующего предмета	успешно
Наличие результата генетического алгоритма	успешно
Наличие мутаций учителей	успешно
Наличие мутаций классов	успешно
Количество особей на этапе размножения	успешно
Число отбираемых после фитнес-функции особей	успешно
Корректность турнирного отбора	успешно
Согласованность данных учеников полученного расписания	успешно
Согласованность данных учителей полученного расписания	успешно
Корректность распределения аудиторий	успешно

Результат выполнения модульных тестов представлен на рисунке ниже, как мы можем заметить, они все успешны.

TestProject (31)	82 мс
TestProject (31)	82 мс
ClassroomTests (7)	72 мс
DeleteClassroom_Current_Succ...	67 мс
DeleteClassroom_nonExist_Thro...	5 мс
ReadClassroom_GetElement_Su...	< 1 мс
ReadClassroom_GetFullList_Suc...	< 1 мс
SaveClassroom_ClassroomExist...	< 1 мс
SaveClassroom_New_Success	< 1 мс
SaveClassroom_NewWithoutCa...	< 1 мс
ClassTests (8)	2 мс
SubjectTests (8)	2 мс
TeacherTests (8)	6 мс

Рисунок 8. Результат выполнения модульного тестирования

В рамках проведения тестирования провалено несколько тестовых сценариев, но они имеют одну проблему – отсутствие ограничения на уникальность поля наименования у объектов в БД. Для исправления данной ошибки необходимо добавить ограничение на уникальность значений

параметра Name для соответствующих таблиц БД и исключение текущих неуникальных записей.

Данная ошибка не была выявлена в рамках модульного тестирования, поскольку ограничение на уникальность отсутствует в БД и при нормальном использовании системы данную ошибку получить невозможно.

Тестирование проводилось 16.03.2023

Приложение 1

Листинг основного проекта